SEMICONDUCTOR DESIGN

Multi-Process Synthesis & Timing Closure

Samsung 32nm · TSMC 180nm

RTL-to-Netlist Implementation & Formal Equivalence

정태웅 · 청주대학교

CONTENTS

Agenda

Problem → analysis → fix → verification, applied across two foundry nodes.

01	**RTL & Foundations**	Code analysis, expected schematics, 3-stage pipeline (TOP), multi-Vt libraries, and the design spec
02	**Samsung 32nm**	Setup, timing & capacitance closure, multi-corner run (main focus)
03	**TSMC 180nm**	Library migration, four-class violations, buffer upsizing, multicycle timing, process limits
04	**Verification & Closing**	Cross-process comparison, datapath reconstruction, Formality, simulation and key takeaways

OVERVIEW

Project Overview

One RTL design synthesized and timing-closed across two nodes, verified at every stage.

2

Foundry Nodes

Samsung 32nm and TSMC 180nm from one shared RTL source

3

Synthesis Runs

Samsung: baseline + WCBC · TSMC: baseline (cap-unit-corrected)

3

Verification Layers

RTL · post-synthesis netlist · multi-corner simulation

PASS

Formality

Formal equivalence confirmed for all variants

OVERVIEW

Common Design Methodology

A unified RTL-to-netlist flow applied identically to both process nodes.



RTL Code Analysis

Interpret the HDL hierarchy, sketch the expected schematic, and locate timing-critical paths.

Topographical Synthesis

compile_ultra with Physical Guidance (-spg) to estimate real interconnect parasitics.

Verification

Netlist + multi-corner simulation and Formality equivalence at each node.

RTL Architecture & Code Analysis

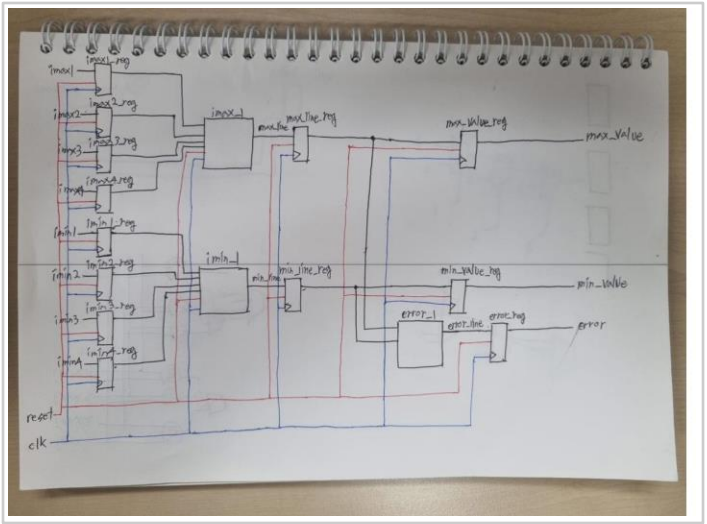
Top module TOP: a pipelined datapath computing max, min and their absolute difference.

Pipeline & Modules

3 pipeline stages (TOP) input regs → mid regs → output regs

maximum → largest of imax1–4
minimum → smallest of imin1–4
error_detector → (larger − smaller), 0 if equal

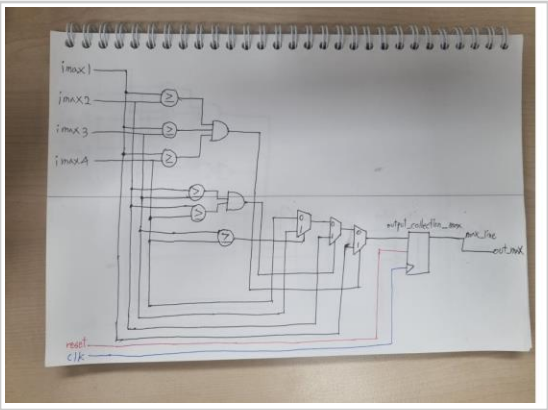
Control clk-synchronous · asynchronous reset
I/O imax1–4, imin1–4 → max_value, min_value, error

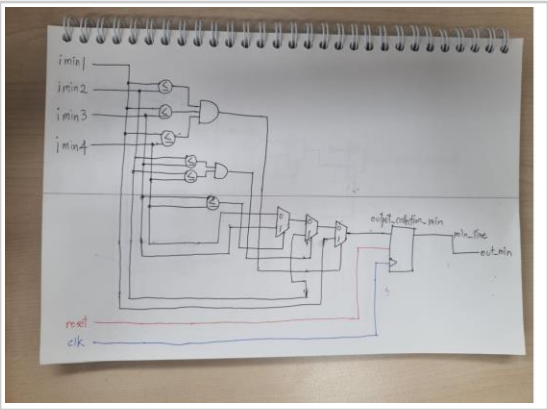


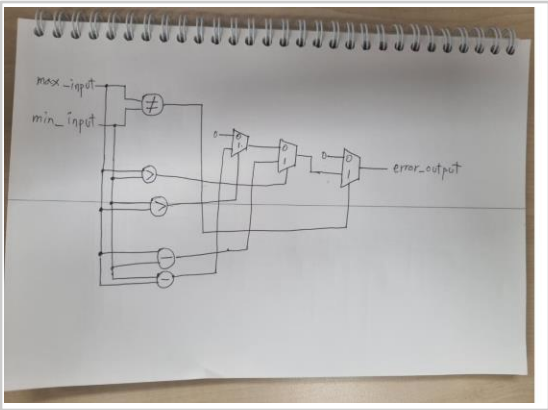
Expected schematic (hand analysis) — verified against the synthesized netlist

Submodule Schematics

Hand analysis of each block — the operators here reappear in Formality's datapath check.





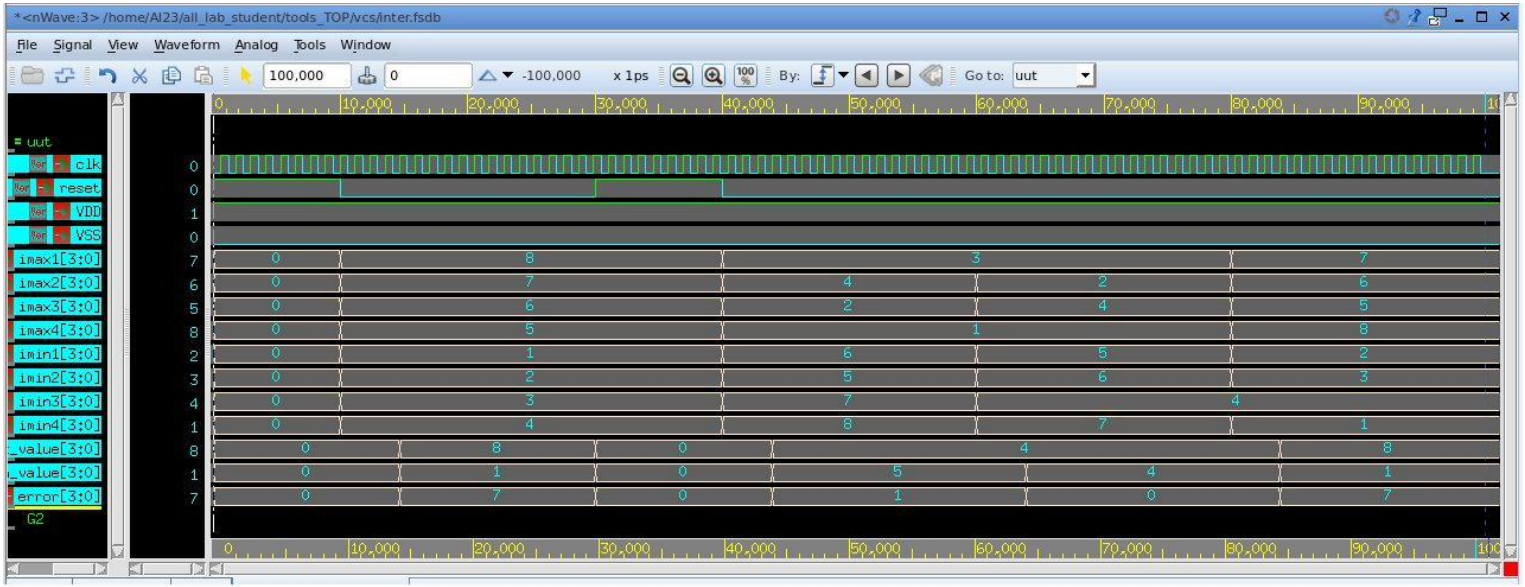


maximum | **minimum** | **error_detector**

maximum / minimum register their output (1 pipeline stage); **error_detector** is combinational, computed between TOP stage 2 → 3. Its ≠, >, −, mux operators reappear in Formality as M_RTL_NEQ / GT / SUB / WMUX.

RTL Functional Verification

Behavioral simulation confirms max / min / error against expected values across the pipeline.



Testbench Verification

Each vector targets a known case — predicted outputs match the simulation exactly.

Phase	imax1–4	imin1–4	max	min	error	What it verifies
reset	0,0,0,0	0,0,0,0	0	0	0	async init
Case 1	8,7,6,5	1,2,3,4	8	1	7	extreme @pos1 · max>min (imax1, imin1)
reset (mid)	8,7,6,5	1,2,3,4	0	0	0	async reset during run
Case 2	3,4,2,1	6,5,7,8	4	5	1	extreme @pos2 · max<min (imax2, imin2)
Case 3	3,2,4,1	5,6,4,7	4	4	0	extreme @pos3 · max==min (imax3, imin3)
Case 4	7,6,5,8	2,3,4,1	8	1	7	extreme @pos4 · max>min (imax4, imin4)

error = | max − min | (0 if equal). Predicted = simulated for every vector — including async reset mid-run and the max>min / max<min / max==min relations.

Multi-Vt Library Background

Why one process ships several libraries — the cap-vs-leakage-vs-speed trade-off.

HVT — high Vth

Hard to switch, slower. Lowest leakage when off.

Low leakage

RVT — regular Vth

The standard balance of speed and leakage.

Baseline

LVT — low Vth

Switches easily, fastest — but higher capacitance and leakage.

High speed

Note Vt is set by channel doping (more doping → higher Vth). The spec caps LVT at 20% (set_multi_vth_constraint), and DC's target_library priority picks the left-most library per Vt group — which is why the worst-case library must be ordered on top (see The Dual-Engine Pitfall).

Design Spec & Constraints

Every constraint traces to the DESIGN1 (TOP) specification — not arbitrary numbers.

Clock

854 MHz / 1.17 ns spec → synthesized at 1.00 ns; uncertainty 0.03 ns, jitter ±10 ps, setup margin 10 ps, latency 0.15 ns.

Design rules & boundary

max_fanout 12 · max_capacitance 2.0 · max_transition 0.12 · LVT cells limited to 20%.
Boundary size {{0 0} {72 72}} · shape rectangle

Output loads

error = 4× MUX21X1/A1 ≈ 1.75 · max_v = 6× MUX41X1/A2 ≈ 2.86 · min_v = 6× NAND2X1/A1 ≈ 2.88.

I/O & driving

Input delay imax/imin 0.76 / 0.05 ns · driving cell IBUFFX4_HVT · output delays 510/400 · 330/350 · 170/200 ps · error[1] multicycle 1.89 ns.

SECTION 02 · SAMSUNG 32NM

Synthesis Setup

compile_ultra tuned for hierarchy preservation, physical guidance, and staged recompiles.

-no_boundary_optimization
Preserve module boundaries

-no_autoungroup
Keep the logical hierarchy intact

-spg
Physical guidance + congestion & layer-aware optimization (topographical only)

-retime
Adaptive retiming to improve delay — tried, then removed (too slow after buffering)

-incr
Skips mapping & implementation-selection; optimizes the existing netlist

-only / -no_design_rule
-no: skip DRC fixing · -only: DRC only (needs -incr)

Commands set_app_var compile_ultra_ungroup_dw false · set_cost_priority -design_rules (Samsung) / -delay (TSMC) · set_svf for equivalence

SECTION 02 · SAMSUNG 32NM

Issue 1 — Setup Timing Violation

Clock period tightened mid-project; existing multicycle constraints no longer matched.

BEFORE

Period 1.17 → 1.00 ns

With the period tightened to 1.00 ns, the original multicycle constraints no longer matched, producing setup violations on the multi-cycle output paths (error / max_value / min_value).

AFTER

Multicycle — spec + experiment

error[1] multicycle is defined by the spec. After the period tightened, I additionally relaxed the other paths (max/min setup 2 / hold 1) — not spec-required, just to try forcing MET.

Setup slack +0.000210 (MET).

Issue 2 — Max Capacitance

Output nets carried abnormal capacitance; some violations are spec-bound and unfixable.

1 **size_cell**

Upsized cells — improved timing only, not capacitance.

2 **insert_buffer**

Reduced net capacitance, but over-insertion created timing slack, so buffers were kept minimal.

3 **only_design_rule recompile**

DRC-only incremental pass; retiming & optimization disabled, so timing stays fixed (WNS ≈ 0) while only the design-rule cost is reduced. Residual = spec-bound max/min_value.

4 **Spec-bound residual**

max_value / min_value loads are ≈3.0 (set by spec) > max_cap 2.0 — structurally impossible to meet.

66 → 8

Max-cap violations (over-cap −41.4 → −8.5)

469 → 285

Buffer / inverter count

1565.8 → 1209.2

Total cell area (μm²)

SECTION 02 · SAMSUNG 32NM

The Dual-Algorithm Pitfall

A 0.78 V design linked a 0.85 V cell — the fix was the target-library order, not a constraint.

The problem

Two algorithm: **mapping** (picks cells) and **timing** (set_operating_conditions). operating_conditions steers **only timing**, so mapping linked a **0.85 V** lib while the design ran at **0.78 V**.

The fix

target_library is priority-ordered — the left-most (top) library is applied first. So putting the **worst-case lib (tt0p78v125c) on top** makes mapping pick it first, aligning both engines on the **same corner**.

set_min_library → which hold (min) library to use — typically a backend/STA concern.
set_operating_conditions → which library's operating_conditions the timing engine loads for delay.

Library Migration

Spec loads were Samsung cells — but the capacitance unit differs, so the load had to be re-scaled.

Item	Samsung saed32	TSMC tcb018
Library header	capacitive_load_unit = ff	capacitance unit = pF
Capacitance unit	fF	**pF (not Samsung's fF)**
Spec loads (error / max_v / min_v)	1.75 / 2.86 / 2.88 — applied directly	**set_load re-scaled to pF → fixed_tsmc**
Driving cell	IBUFFX4_HVT	BUFFD2 (-library tcb018g3d3tc)
Timing / transition	unit-independent	unit-independent — unchanged

Migration story

1 Applied the spec loads directly, as on Samsung.

2 Checked the TSMC library — capacitance unit is pF, not Samsung's fF.

3 Re-scaled set_load and re-ran (timing/transition are unit-independent) → fixed_tsmc.

Loads & Driving

Loads error 1.75 · max_v 2.86 · min_v 2.88

Driving set_driving_cell -library tcb018g3d3tc -lib_cell BUFFD2 (set_drive / set_input_transition tried first, inaccurate).

SECTION 03 · TSMC 180NM

Violation Landscape

The 180nm process produced four distinct violation classes — far more than at 32nm.

Fanout

Nets above max_fanout 12 across the comparison logic — fixed by report_net-guided buffer splitting.

Timing

Setup and hold both failing — the slower 180nm cells miss the 1.0 ns target.

Transition

Large transitions on error-path nets driven by weak buffers.

Capacitance

Output loads above max_cap 2.0, worsened by tight layout space.

Buffer Upsizing — BUFFD6 → BUFFD16

The error cap limit was the driver's own attribute, not a user constraint.

BUFFD6 · max_capacitance

1.982

size_cell →

BUFFD16 · max_capacitance

5.286

Item	BUFFD6 (before)	BUFFD16 (after size_cell)
Pin Z max_capacitance (pF)	1.982	5.286
Source of the limit	driver's own lib attribute — not a user constraint	driver's own lib attribute — ceiling raised
error capacitance violation	required capacitance 1.982 — VIOLATED	cleared
Transition violation	VIOLATED	cleared (larger drive)

Why it works

The violation's "required capacitance 1.982" was BUFFD6's own pin-Z max_capacitance attribute — not a constraint I set. Swapping to BUFFD16 raises the ceiling to 5.286, clearing the error capacitance violation and, with the larger drive, the transition violation as well.

Multicycle — An Experiment

Not required by the spec — applied to see how much multicycle relaxes the output paths.

```
Startpoint: clk_r_REG21_S4
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: error[0] (output port clocked by clk)
Path Group: clk
Path Type: max

Point                                   Incr       Path
-----------------------------------------------------------
clock clk (rise edge)                   0.000000   0.000000
clock network delay (ideal)             0.240000   0.240000
clk_r_REG21_S4/CP (DFCNQD4)             0.000000   0.240000 r
clk_r_REG21_S4/Q (DFCNQD4)              0.342791   0.582791 r
U8/Z (BUFFD16)                          0.429986 * 1.012778 r
error[0] (out)                          0.034484 * 1.047262 r
data arrival time                                  1.047262

clock clk (rise edge)                   1.000000   1.000000
clock network delay (ideal)             0.240000   1.240000
clock uncertainty                      -0.050000   1.190000
output external delay                  -0.610000   0.580000
data required time                                 0.580000
-----------------------------------------------------------
data required time                                 0.580000
data arrival time                                 -1.047262
-----------------------------------------------------------
slack (VIOLATED)                                  -0.467262
```

No multicycle — error[0] slack −0.467 (VIOLATED)

```
Startpoint: clk_r_REG14_S4
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: max_value[1]
            (output port clocked by clk)
Path Group: clk
Path Type: max

Point                                   Incr       Path
-----------------------------------------------------------
clock clk (rise edge)                   0.000000   0.000000
clock network delay (ideal)             0.240000   0.240000
clk_r_REG14_S4/CP (DFCNQD2)             0.000000   0.240000 r
clk_r_REG14_S4/Q (DFCNQD2)              0.338506   0.578506 r
U8/Z (BUFFD16)                          0.607835 * 1.186341 r
max_value[1] (out)                      0.053092 * 1.239433 r
data arrival time                                  1.239433

clock clk (rise edge)                   2.000000   2.000000
clock network delay (ideal)             0.240000   2.240000
clock uncertainty                      -0.050000   2.190000
output external delay                  -0.950000   1.240000
data required time                                 1.240000
-----------------------------------------------------------
data required time                                 1.240000
data arrival time                                 -1.239433
-----------------------------------------------------------
slack (MET)                                        0.000567
```

With multicycle — max_value[1] slack +0.000567 (MET)

error setup 4 / hold 3, max & min setup 3 / hold 2 — not a spec requirement; an experiment to observe how far multicycle relaxes the output paths.

SECTION 03 · TSMC 180NM

Process Limitation & Trade-off

After the cap-unit fix, prioritizing timing trades off DRC and area — the real 180nm wall.

Output (multicycle)

MET

max / min / error paths

Worst setup

−0.246

w as −0.328 · internal S2→S3

Hold

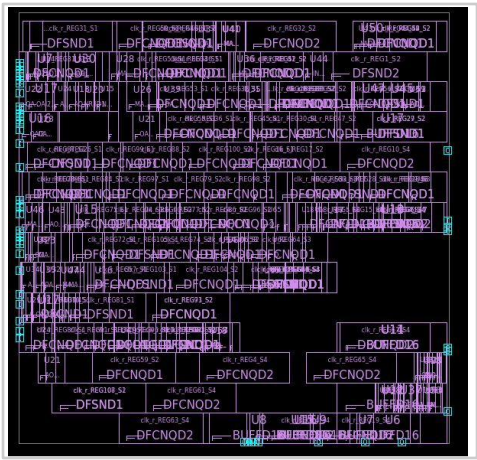
−0.064

w orst-case residual

DRC (unit-fixed)

tr 583 · cap 1195

transition & cap w alls



Layout — no room left to insert buffers

Engineering judgment cost -delay improved setup (−0.328→−0.246) but raised transition/cap & area (21,975→24,930 μm² · 76.5 mW). Formality PASSED.

Cross-Process Comparison

The same RTL and the same fixes behave very differently across the two nodes.

	Samsung 32nm	TSMC 180nm
Synthesis	spg · retime · cost=DR	migrated · spg · retime · cost=delay
Key issue	Setup timing & max cap	Fanout / Trans / Cap + timing
Main fix	Multicycle + min buffers	Net split + BUFFD6→16 upsizing
Cell area	1209 (↓ from 1566)	24,930 (↑ from 21,975)
Setup	+0.0002 MET	output MET / internal −0.246
Power	0.51 mW	76.5 mW
Formality	PASS	PASS

Common to both: -no_autoungroup · -no_boundary_optimization · compile_ultra_ungroup_dw=false. Optimization reduced area at 32nm but increased it at 180nm — the same fixes pay off differently by node.

SECTION 04 · VERIFICATION & CLOSING

Why the Nodes Differ — Process Effects

Same RTL; the 32nm ↔ 180nm gap comes from fundamental process scaling.

Feature size

~5.6× larger (32 → 180 nm)

Bigger cells → area 1,209 vs 24,930 µm²

Transistor speed

Smaller node switches faster

32nm clears 1 ns; 180nm setup stays tight (−0.246)

Capacitance

pF-scale loads & higher parasitics

cap 1195 · transition 583 violations at 180nm

Power & layout

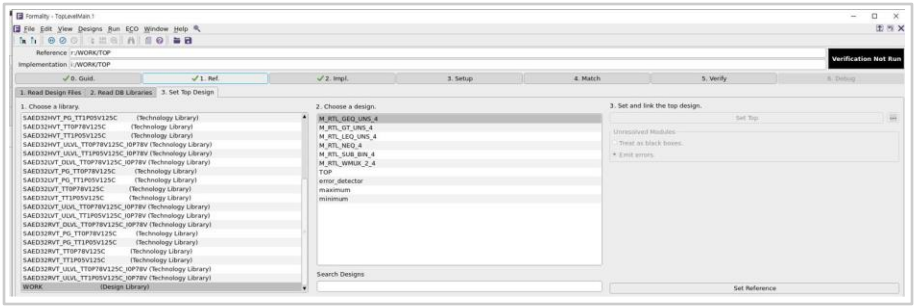
More energy/gate · fewer metal layers

0.5 vs 76.5 mW; no room to insert buffers

Bottom line the same design and the same fixes — the node's physics sets the ceiling on area, power and timing.

Why the Reference Shows Extra Designs

Loading RTL into Formality's reference creates designs that aren't written in the source.



Formality reference — M_RTL_GEQ / GT / LEQ / NEQ / SUB_BIN / WMUX appear beside TOP, maximum, minimum, error_detector

Not an error — datapath reconstruction

The RTL operators (≠, >, ≥, ≤, −, mux) are extracted as separate datapath designs so the reference matches the netlist's datapath optimizations.

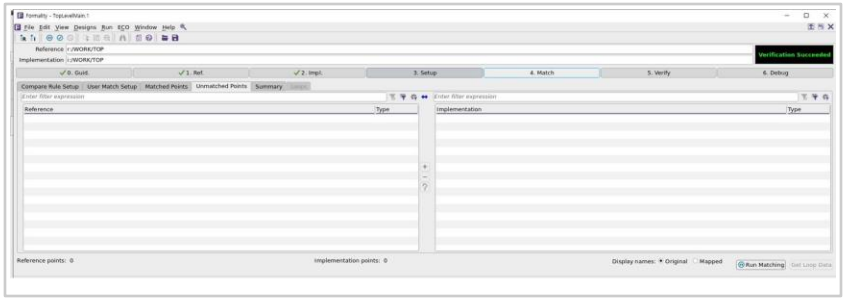
Three Formality techniques

Restructuring (uses the SVF to mirror DC's transforms), a special datapath solver (matches operators and removes them from the check), and other solvers.

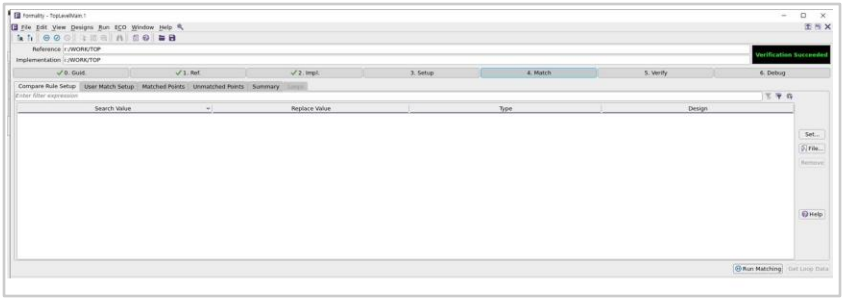
Reference: “RTL Design Verification by Making Use of Datapath Information”

Formality Equivalence Check

RTL ≡ netlist verified on both nodes; minimizing buffers was key to a clean comparison.



Samsung 32nm — Verification Succeeded



TSMC 180nm — Verification Succeeded

All variants PASS Samsung 32nm · Samsung WCBC · TSMC 180nm. A buffer-heavy netlist first failed (suspected SVF mismatch); minimizing inserted buffers and retiming restored equivalence.

Retime Hides the Block Hierarchy

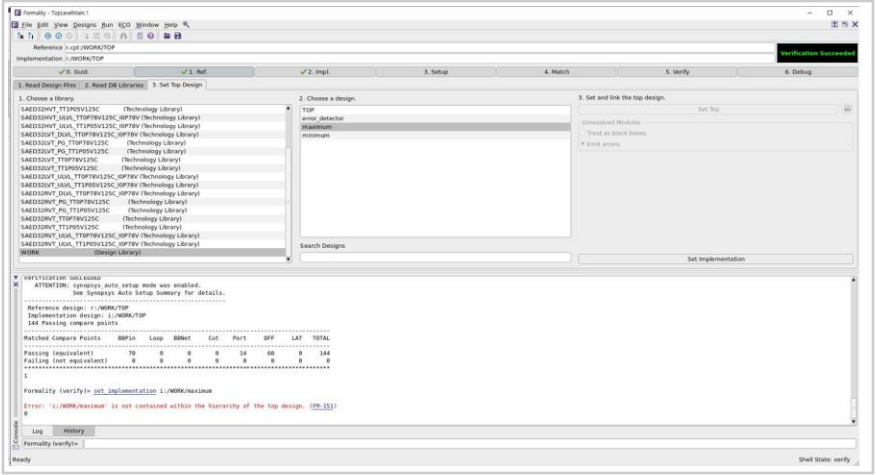
Hierarchy-preserving flags were set and the blocks survive — yet fm_shell can't resolve them after retime.

Flags set · blocks survive

-no_autoungroup · -no_boundary_optimization · compile_ultra_ungroup_dw = false; 328 retiming guides still relocate registers across boundaries. maximum / minimum / error_detector remain in the netlist.

report_matched_points evidence

Impl regs sit flat under TOP (i:/WORK/TOP/clk_r_REG10_S4) with asymmetric fm_ret_* wrappers — imax_1 / maximum no longer contain them, so set_implementation i:/WORK/maximum → FM-151.

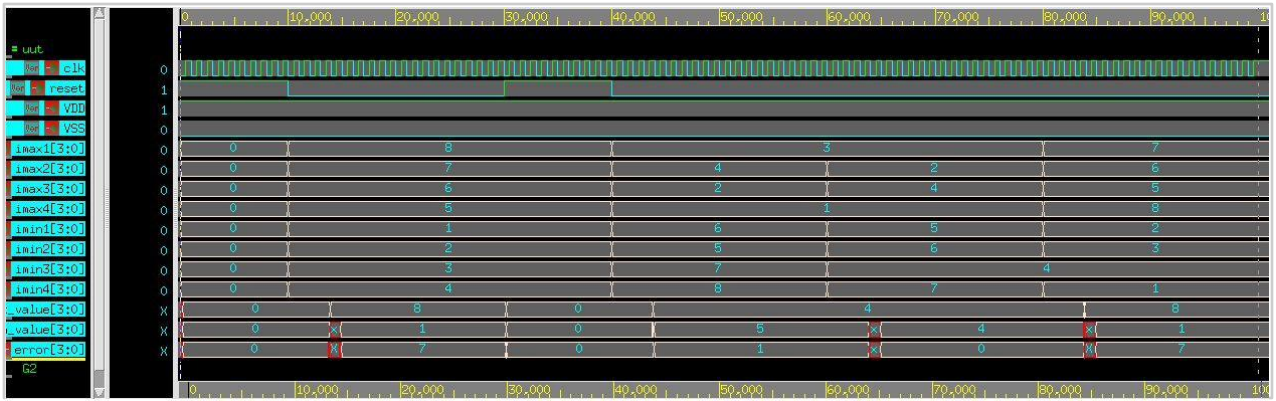


fm_shell — FM-151 when setting impl top to maximum

Equivalence is unaffected Formality verifies on the TOP boundary — Verification SUCCEEDED; only fm_shell's block recognition is lost, not the logic.

Netlist Simulation · Single-corner

Gate-level netlist on the worst-case library (no multi-corner setup yet) reproduces the RTL response.



Single-corner netlist simulation (baseline)

+0.0002

Setup MET (baseline)

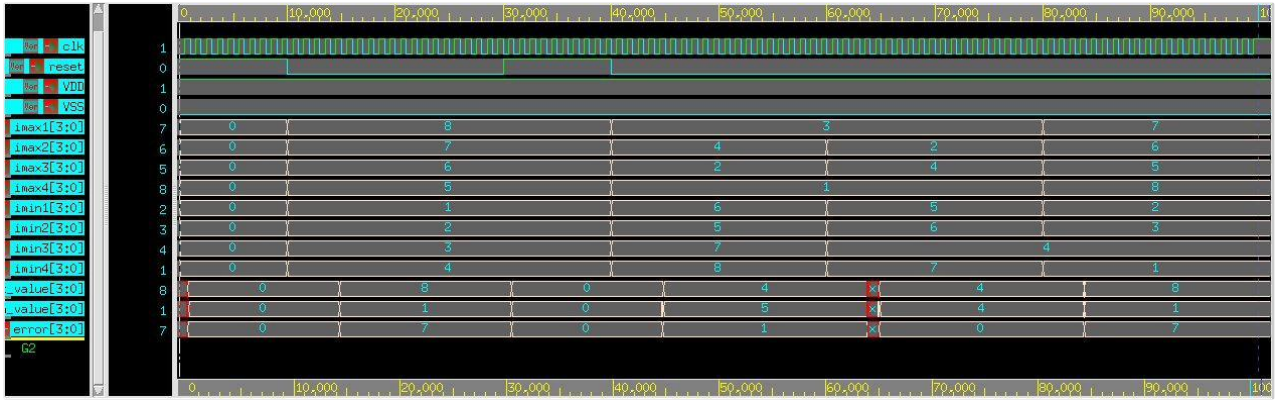
511.8 µW

Power (baseline)

Note Segments that look like don't-cares (X) are gate-level cell-delay artifacts, not functional mismatches.

Netlist Simulation · Multi-corner

set_min_library (WCBC) applied so setup·hold are judged across best/worst corners.



Multi-corner netlist simulation (WCBC)

+0.0104

Setup slack (MET) at corner

714.7 µW

Power, up from 511.8 — corner trade-off

In practice the frontend signs off setup on the worst-case library; the hold / min-corner is a backend task — checked here for completeness.

SECTION 04 · VERIFICATION & CLOSING

Key Takeaways

01

Process-Aware Constraints

Each node needed its own loads, driving cell, multicycle depth, and library order.

02

Physical-Guided Optimization

Topographical synthesis and GUI/layout inspection drove the real capacitance fixes.

03

Verification

RTL, netlist and multi-corner simulation plus Formality gave end-to-end confidence.

04

Engineering Judgment

Prioritizing timing over exhaustive DRC closure was a realistic 180nm trade-off.

Next steps free layout space via floorplan-aware buffering · evaluate LVT on the internal S2→S3 path · close the residual 180nm transition/cap.